embarca tech | Residência Tecnológica em Sistemas Embarcados

SBC Labrador – Parte 3

RESIDÊNCIA – Aula Síncrona (14/10/2025)

Prof. Dr Ricardo Menezes Prates

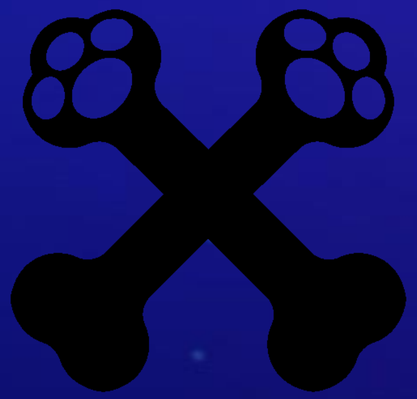
Executores:
INSTITUTO FEDERAL Piauí
INSTITUTO FEDERAL Rio Grande do Norte
INSTITUTO FEDERAL Maranhão
INSTITUTO FEDERAL Ceará
cepedi
INSTITUTO HARDWARE BR
Coordenação: Softex
Iniciativa: MINISTÉRIO DA CIÊNCIA, TECNOLOGIA E INOVAÇÃO
GOVERNO FEDERAL BRASIL UNIÃO E RECONSTRUÇÃO

- ***Introdução***
  - Planejamento da aula

- **Computador Labrador – Single Board Computer (SBC)**
  - *Interfaces de comunicação: UART, I2C*
  - *Conversor A/D*

- **Conclusões**

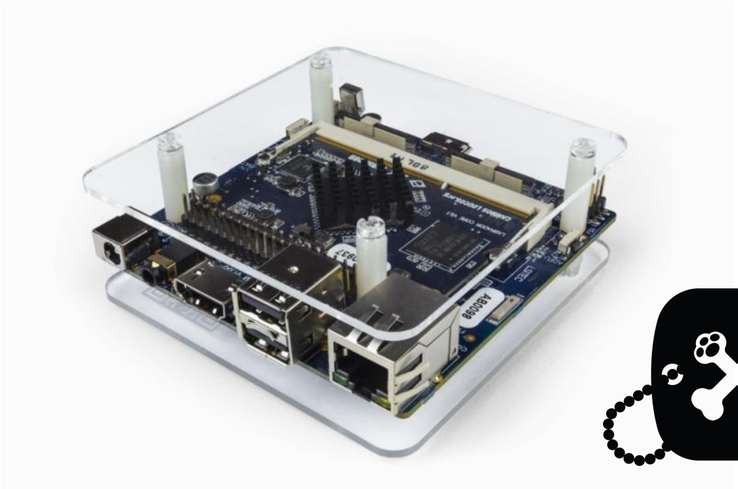
Introdução



Objetivo

- **Objetivo**: Programação de periféricos da SBC Labrador: Conversor Análogico Digital (A/D ou ADC) e Interfaces de comunicação UART e I2C.

- Avaliação dos resultados através da linguagem de programação C, da IDE Code::Clocks e do Sistema de Análise de Sinais **PulseView/sigrok**.



Placa Labrador

PulseView

Planejamento da Aula

- **Pré-requisitos para esta aula**:

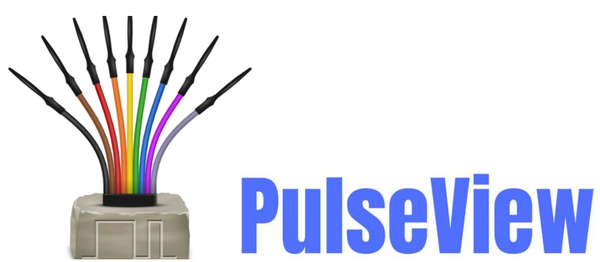
✓ Linguagem C avançada;

✓ *Interface de comunicação – UART e I2C;*

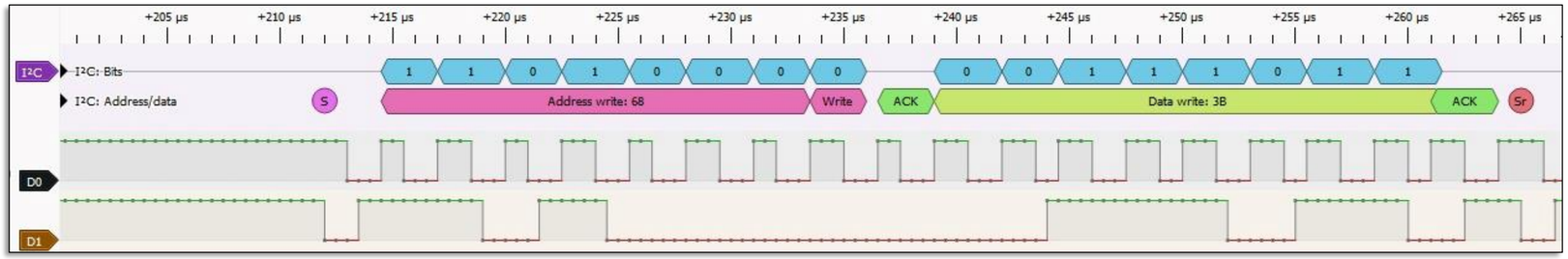
✓ *Analisadores Lógicos;*

✓ *Conversor A/D;*

✓ *Linux básico.*



PulseView



Planejamento da Aula

- **Potenciais perguntas a serem trabalhadas nesta aula.**

✓ É possível utilizar o conversor A/D na placa labrador utilizando a linguagem C?

✓ Como trabalho com as interfaces de comunicação UART e I2C com os periféricos presentes na placa labrador?

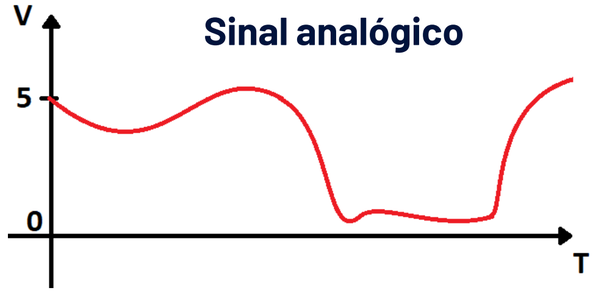
✓ É possível obter e analisar informações de sensores I2C com a SBC labrador?

Labrador - Periféricos

CANÍNOS LOUCOS .org

Conversor ADC – Revisão

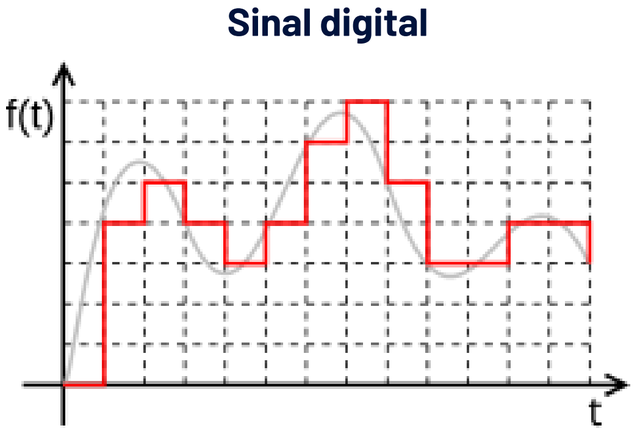
- **Conversão Analógica-Digital (ADC):**



✓ A **Conversão Analógica-Digital (ADC)** é o processo de converter sinais analógicos em sinais digitais.

✓ **Conversor ADC**: Dispositivo que converte sinais analógicos (variáveis contínuas) em sinais digitais (variáveis discretas).

✓ Os **microcontroladores e microprocessadores** precisam do ADC para tratar dados de sensores analógicos, pois só lidam com sinais digitais.



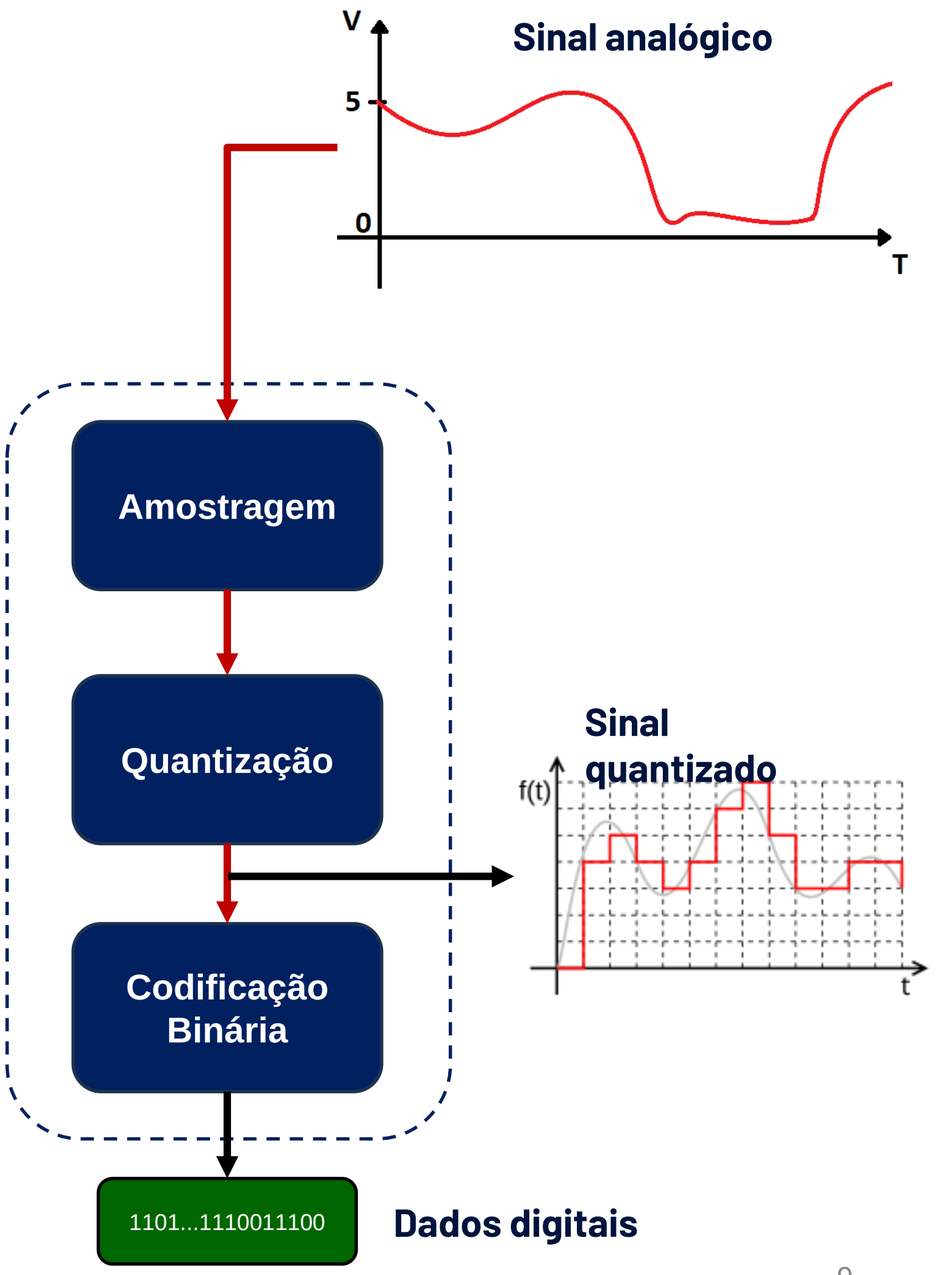
Conversor ADC – Revisão

- **Princípios de funcionamento:**

✓ 1. **Amostragem**: o sinal analógico é amostrado em intervalos regulares.

✓ 2. **Quantização**: o sinal amostrado é convertido em um valor digital aproximado.

✓ 3. **Codificação**: o valor digital é codificado em um formato binário.



Conversor ADC – Revisão

- **Características da conversão AD:**

✓ 1. **Resolução**: Define a precisão do ADC – 8 bits, 10 bits, 12 bits etc. → $\text{Resolução} = \frac{V_{ref}}{2^n}$

✓ 2. **Taxa de amostragem**: Número de amostras feitas por segundo.

✓ 3. **Tensão de referência**: Intervalo de tensões que o conversor ADC pode medir.

Conversor ADC – Revisão

- **Características da conversão AD:**

✓ 1. **Resolução**: Define a precisão do ADC – 8 bits, 10 bits, 12 bits etc. → $\text{Resolução} = \frac{V_{ref}}{2^n}$

✓ 2. **Taxa de amostragem**: Número de amostras feitas por segundo.

✓ 3. **Tensão de referência**: Intervalo de tensões que o conversor ADC pode medir.

- Ex: Para o RP2040:

$$\text{Resolução} = \frac{3{,}3V}{2^{12}} = \frac{3{,}3V}{4096} = 805{,}6 \times 10^{-6}\ \text{V/bit}$$

Conversor ADC – Revisão

- **Características da conversão AD:**

✓ 1. **Resolução**: Define a precisão do ADC – 8 bits, 10 bits, 12 bits etc.

✓ 2. **Taxa de amostragem**: Número de amostras feitas por segundo.

✓ 3. **Tensão de referência**: Intervalo de tensões que o conversor ADC pode medir.

Menor Valor Detectável (LSB)

Número de bits	Níveis	Largura do passo (3,3V)
8-bits	256	12,89 mV
10-bits	1024	3,22 mV
12-bits	4096	805 uV
16-bits	65536	50,3 uV
18-bits	262144	12,6 uV
20-bits	1048576	3,15 uV
24-bits	16777216	197 nV

Conversor ADC

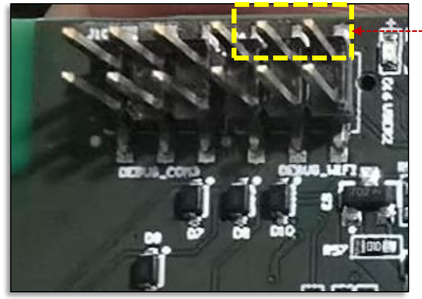
- **Labrador –** /sys/kernel/auxadc**:**

✓ 1. **O que é**: Faz parte da interface de drivers auxiliares do kernel do Linus usada pela caninos loucos.

✓ 2. **Uitilidade**: Ele dá acesso direto ao hardware ADC através de um driver chamado de auxadc.

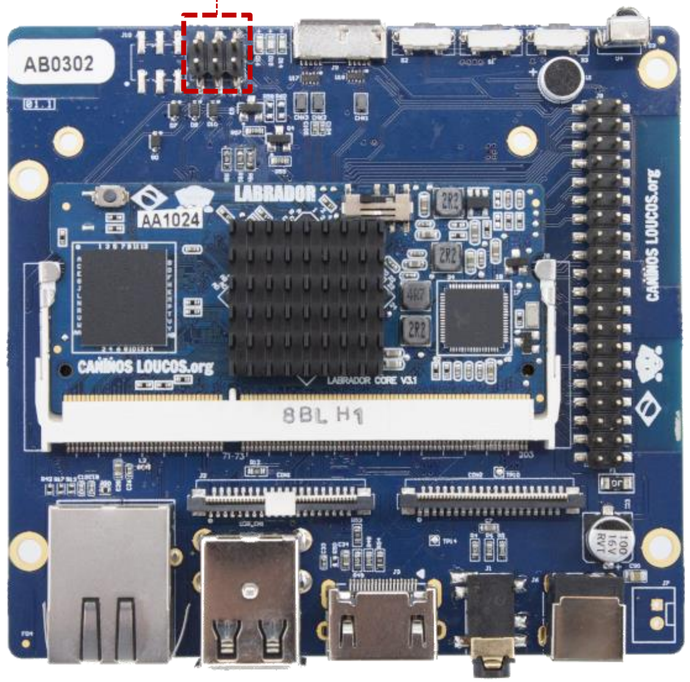
✓ 3. **Dados obtidos:** valores brutos de conversão.

Labrador - EmbarcaTech:



J4 - Labrador

TX3	1	2	ADC0_COM
RX3	3	4	ADC0
GND	5	6	GND



Conversor ADC

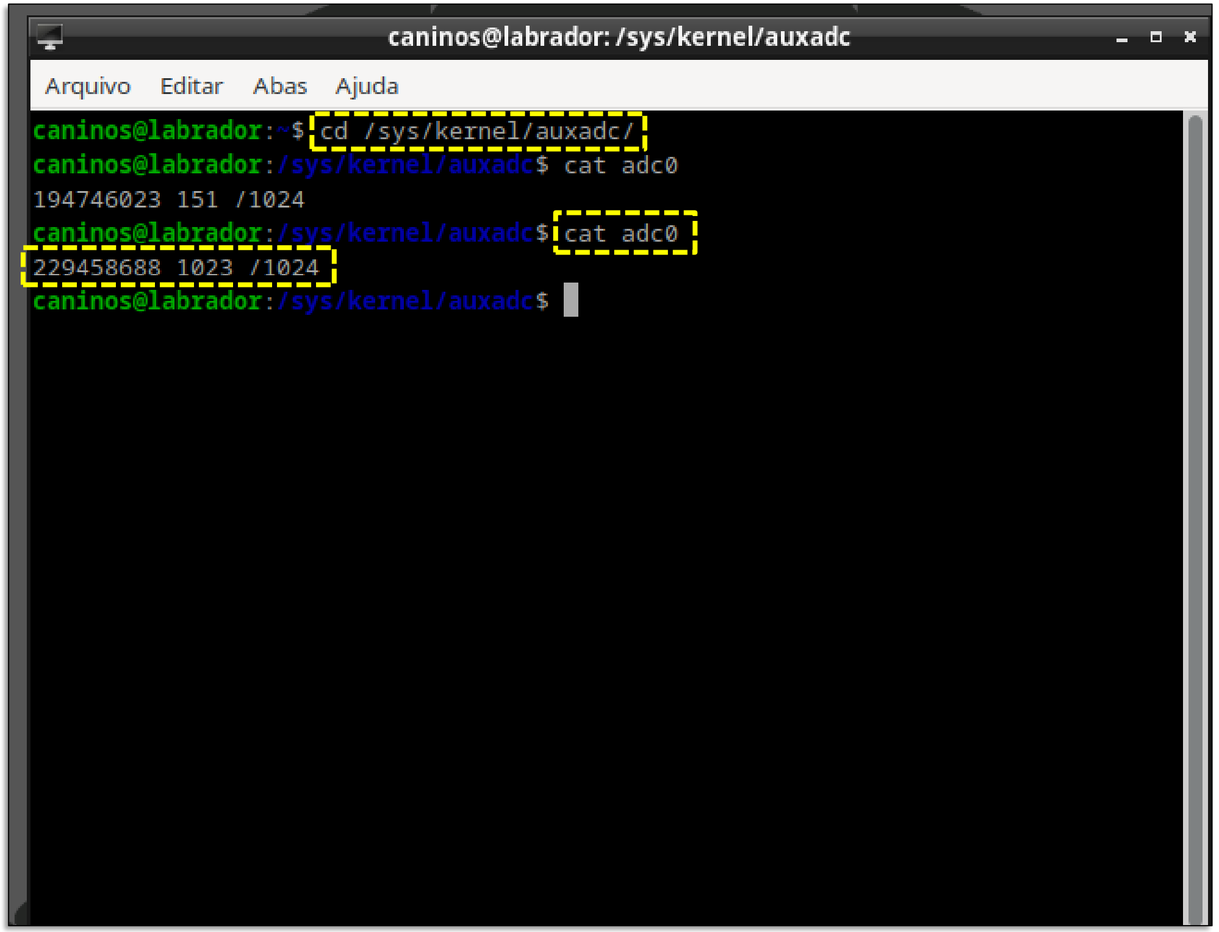
- **Labrador –** /sys/kernel/auxadc**:**

✓ 1. **O que é**: Faz parte da interface de drivers auxiliares do kernel do Linus usada pela caninos loucos.

✓ 2. **Uitilidade**: Ele dá acesso direto ao hardware ADC através de um driver chamado de auxadc.

✓ 3. **Dados obtidos:** valores brutos de conversão.

Terminal Linux:



Conversor ADC

Código em C:

```c
#include <stdio.h>          // Funções de entrada/saída: fopen, fscanf, printf, perror
#include <stdlib.h>         // Funções utilitárias gerais (exit, malloc, etc.)
#include <unistd.h>         // Funções POSIX: usleep(), close(), etc.

// Função principal
int main() {
    FILE *fp;  //ponteiro para arquivo, usado para acessar o ADC.
    unsigned long timestamp = 0;
    int value = 0, scale = 0;
    const char *adc_path = "/sys/kernel/auxadc/adc0";

    // Loop de leitura contínua
    while (1) {
        /*Abre o arquivo /sys/kernel/auxadc/adc0 em modo leitura ("r")
        O kernel fornece uma leitura “instantânea” (snapshot) do ADC.
        Caso o driver ou o caminho não exista, fopen() retorna NULL, e o erro é exibido com
perror().
        */
        fp = fopen(adc_path, "r");
        if (fp == NULL) {
            perror("Erro ao abrir ADC");
            return 1;
        }
        /*
        Lê formato: <timestamp> <value> / <scale>
        fscanf() lê os três valores numéricos do arquivo aberto.
        */
        if (fscanf(fp, "%lu %d / %d", &timestamp, &value, &scale) == 3) {
            float voltage = (value * 3.3f) / scale;
            printf("ADC0 → tempo=%lu | valor=%d | escala=%d | tensão=%.3f V\n",
                   timestamp, value, scale, voltage);
        } else {
            printf("Formato inesperado do ADC\n");
        }

        fclose(fp);
        usleep(200000); // lê a cada 200 ms
    }

    return 0;
}
```

Conversor ADC

Resultado:

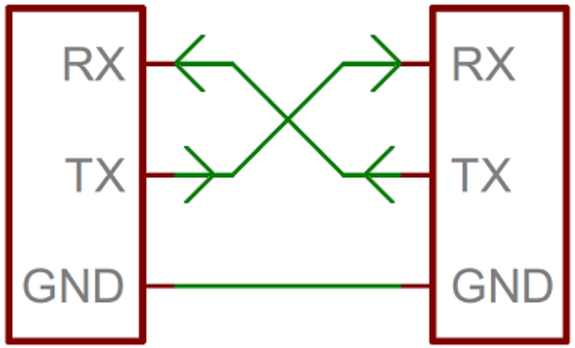
adc_01

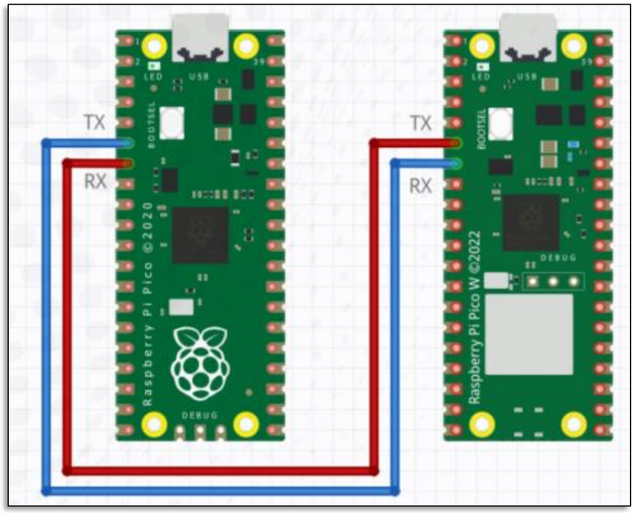
Arquivo Editar Abas Ajuda

```
ADC0 → tempo=559500774 | valor=1023 | escala=1024 | tensão=3.297 V
ADC0 → tempo=559701353 | valor=1023 | escala=1024 | tensão=3.297 V
ADC0 → tempo=559902160 | valor=1023 | escala=1024 | tensão=3.297 V
ADC0 → tempo=560103077 | valor=1023 | escala=1024 | tensão=3.297 V
ADC0 → tempo=560308846 | valor=1023 | escala=1024 | tensão=3.297 V
ADC0 → tempo=560509803 | valor=1023 | escala=1024 | tensão=3.297 V
ADC0 → tempo=560710704 | valor=1023 | escala=1024 | tensão=3.297 V
ADC0 → tempo=560911474 | valor=1023 | escala=1024 | tensão=3.297 V
ADC0 → tempo=561112223 | valor=1023 | escala=1024 | tensão=3.297 V
ADC0 → tempo=561313122 | valor=1023 | escala=1024 | tensão=3.297 V
ADC0 → tempo=561514187 | valor=1023 | escala=1024 | tensão=3.297 V
ADC0 → tempo=561715160 | valor=1023 | escala=1024 | tensão=3.297 V
ADC0 → tempo=561916288 | valor=1023 | escala=1024 | tensão=3.297 V
ADC0 → tempo=562117228 | valor=1023 | escala=1024 | tensão=3.297 V
ADC0 → tempo=562318514 | valor=1023 | escala=1024 | tensão=3.297 V
ADC0 → tempo=562519391 | valor=1023 | escala=1024 | tensão=3.297 V
ADC0 → tempo=562720097 | valor=1023 | escala=1024 | tensão=3.297 V
ADC0 → tempo=562920746 | valor=1023 | escala=1024 | tensão=3.297 V
ADC0 → tempo=563121398 | valor=1023 | escala=1024 | tensão=3.297 V
ADC0 → tempo=563322044 | valor=1023 | escala=1024 | tensão=3.297 V
ADC0 → tempo=563522689 | valor=1023 | escala=1024 | tensão=3.297 V
ADC0 → tempo=563723451 | valor=1023 | escala=1024 | tensão=3.297 V
ADC0 → tempo=563924115 | valor=1023 | escala=1024 | tensão=3.297 V
```

UART - Revisão

- ✓ **Funcionamento**: O pino de transmissão (Tx) do protocolo envia um pacote de bits que será interpretado bit a bit pelo pino receptor.
- ✓ Cada pacote enviado contém 1 start bit que indica o início da mensagem, 1 ou 2 stop bits para indicar o final da mensagem, 5 a 9 bits de informação e 1 bit de paridade para evitar a recepção de erros.
- ✓ **Características**: Protocolo assíncrono, full-duplex, simples de ser implementado.
- ✓ **Forma de ligação**: a comunicação é feita por dois pinos Rx/Tx que dependem do *baud rate* como referência.





UART - Labrador

✓ **Configuração - terminal:**

Opção 1:

$ sudo stty -F /dev/ttyS0 115200

Opção 2:

$ sudo stty -F /dev/ttyS0 115200 cs8 -cstopb -parenb -echo

Localização dos pinos:

FUNC	SODIMM	HEADER		SODIMM	FUNC
3.3V	N/C	1	2	N/C	5V
GPIOE3/TWI_SDATA	41	3	4	N/C	5V
GPIOE2/TWI2_SCLK	43	5	6	N/C	GND
GPIOB18/OAP/TS_CLK/LCD0_D19	134	7	8	17	**GPIOC27**/SPI1_SS/I2C0_SCLK/SPDIF/UART0_TX
GND	N/C	9	10	19	**GPIOC26**/SPI1_MISO/I2C0_SDATA/UART0_RX
GPIOC0/DSI_DP3/SD1_CLK/LCD0_D16	161	11	12	34	**GPIOB8**/PWM3/SD0_CLK/KS_OUT1
GPIOC1/DSI_DN3/SD1_D3/LCD0_D9	163	13	14	N/C	GND
GPIOC4/DSI_CP/SD1_D1/LCD0_D1	167	15	16	103	**GPIOA25**/SIRQ1
3.3V	N/C	17	18	149	**GPIOC6**/DSI_DP0/UART2_RX/SPI0_MISO
GPIOC25/TWI3_SDATA/PCM0_SYNC/SPI0_MOSI	53	19	20	N/C	GND
GPIOC24/I2S_MCLK1/PCM0_IN/SPI0_MISO	55	21	22	169	**GPIOC5**/LCD0_D0/DSI_CN/SD1_D0
GPIOC22/TWI3_SCLK/PCM0_CLK/SPI0_SCLK	47	23	24	51	**GPIOC23**/I2S_LRCLK1/PCM0_OUT/SPI0_SS
GND	N/C	25	26	136	**GPIOB19**/OAN/TS_START/LCD0_D15
GPIOB16/OBP/TS_IN1/LCD0_D21	140	27	28	146	**GPIOB14**/OCP/TS_IN0/LCD0_D23
GPIOB15/OCN/TS_IN2/LCD0_D22	148	29	30	N/C	GND
GPIOB10/OEP/TS_IN7/LCD0_DCLK0	158	31	32	154	**GPIOB13**/ODN/TS_IN4/LCD0_VSYNC0
GPIOB0/PCM0_OUT/I2S_BCLK1	9	33	34	N/C	GND
GPIOB1/PCM0_CLK/I2S_LRCLK1	13	35	36	7	**GPIOA28**/PCM0_IN/I2S_BCLK0
GPIOB2/PCM0_SYNC/I2S_MCLK1	15	37	38	5	**GPIOA31**/I2S_D1
GND	N/C	39	40	3	**GPIOA27**/I2S_D0

UART - Labrador

✓ **Enviar a palavra:**

$ echo "teste" | sudo tee /dev/ttyS0

Obs.: Quando você manda uma string com echo "Teste" por exemplo, o terminal geralmente adiciona um enter no final, que pode virar:

- Teste[0D][0A] - Esses valores são códigos hexadecimais da tabela ASCII:

0D (hex) = 13 (decimal) → Carriage Return (CR)

0A (hex) = 10 (decimal) → Line Feed (LF)

PulseView

UART - Labrador

✓ **Enviar a palavra:**

$ echo "teste" | sudo tee /dev/ttyS0

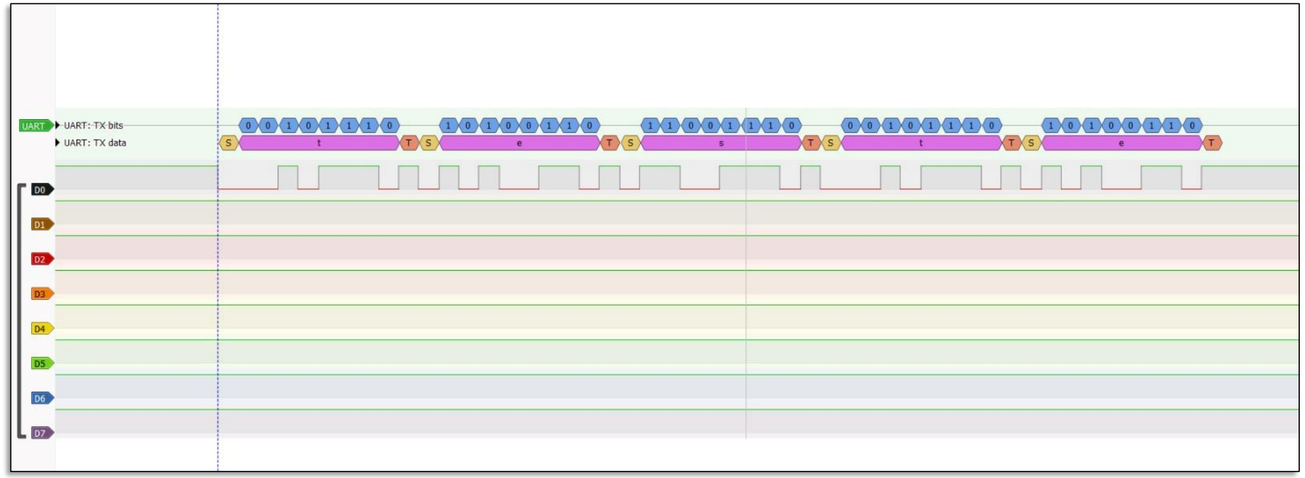
Obs.: Quando você manda uma string com echo "Teste" por exemplo, o terminal geralmente adiciona um enter no final, que pode virar:

- Teste[0D][0A] - Esses valores são códigos hexadecimais da tabela ASCII:

0D (hex) = 13 (decimal) → Carriage Return (CR)

0A (hex) = 10 (decimal) → Line Feed (LF)

Obs.2.: Se você usar echo -n "Teste", aí só vai Teste sem os terminadores.



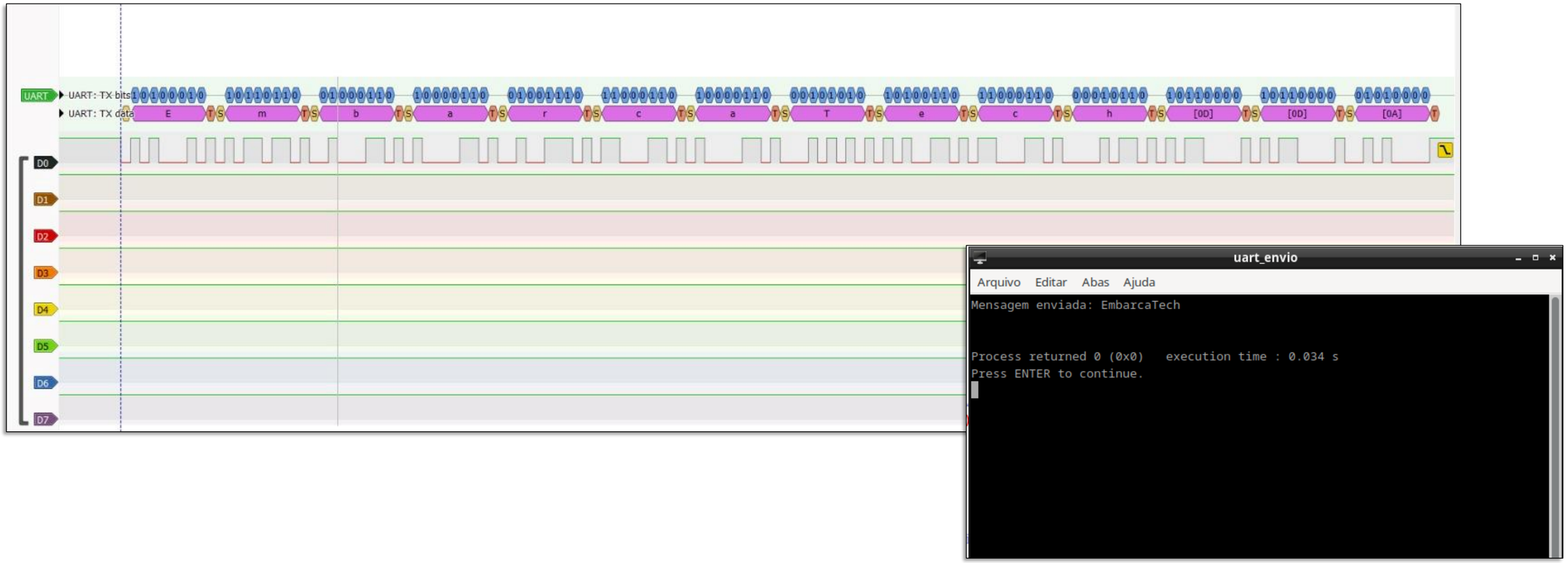
UART - Labrador

Código em C:

```c
#include <stdio.h>      // Funções de entrada/saída
#include <stdlib.h>     // Funções utilitárias (exit, malloc)
#include <unistd.h>     // Funções POSIX (read, write, close)
#include <fcntl.h>      // Controle de arquivos (open, O_RDWR, O_NOCTTY, etc.)
#include <termios.h>    // Controle de terminais (UART, configurações seriais)
#include <string.h>     // Manipulação de strings (strlen)

// Função principal
int main() {

    int fd;
    struct termios options;


    // Abre a UART (ajuste para a porta correta, ex: /dev/ttyS0 ou /dev/ttyUSB0)
    // O_RDWR abre para leitura e escrita.
    // O_NOCTTY impede que este terminal se torne o terminal de controle do processo.
    // O_NDELAY faz o open() retornar imediatamente, mesmo que nenhuma conexão serial esteja pronta.
    fd = open("/dev/ttyS0", O_RDWR | O_NOCTTY | O_NDELAY);
    if (fd == -1) {
        perror("Erro ao abrir a UART");
        return 1;
    }

    // Configuração da UART
    tcgetattr(fd, &options);              // lê a configuração atual
    cfsetispeed(&options, B115200);       // Define baud rate de entrada - UART é configurada com 115200 bps
    cfsetospeed(&options, B115200);       // Define baud rate de saída
    options.c_cflag &= ~PARENB;           // Desativa paridade
    options.c_cflag &= ~CSTOPB;           // 1 stop bit
    options.c_cflag &= ~CSIZE;            // limpa bits anteriores
    options.c_cflag |= CS8;               // 8 bits de dados
    options.c_cflag |= (CLOCAL | CREAD);  // habilita recepção de dados

    tcsetattr(fd, TCSANOW, &options);     // aplica as configurações imediatamente

    // Envio da mensagem pela UART
    char msg[] = "EmbarcaTech\r\n";       // \r = 0D, \n = 0A
    int n = write(fd, msg, strlen(msg));  // Envia a mensagem

    // Caso ocorre erro no envio
    if (n < 0) {
        perror("Erro ao escrever na UART");
    } else {
        printf("Mensagem enviada: %s\n", msg);
    }

    close(fd);
    return 0;
}
```

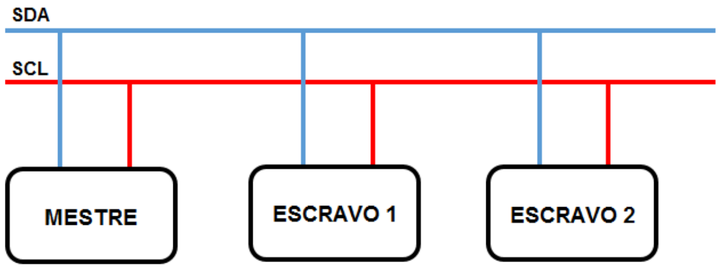
UART - Labrador

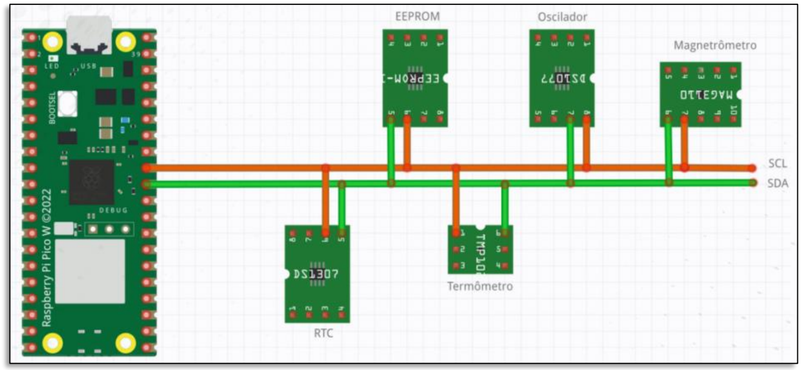
Resultado:



I2C - Revisão

- ✓ **TWI, IIC ou I2C** foi projetado pela Philips Semiconductors, é um protocolo de barramento serial com dois fios.
- ✓ **Características**: A comunicação I2C utiliza apenas dois fios: SCL (serial clock) e SDA (serial data - dados) – **Half duplex**.
- ✓ **Mestre-escravo**: A comunicação I2C é baseada em um modelo mestre-escravo, onde um dispositivo mestre controla a comunicação e os dispositivos escravos respondem aos comandos do mestre.
- ✓ **Endereçamento**: Cada dispositivo I2C tem um endereço único que é utilizado para identificar o dispositivo e realizar a comunicação.



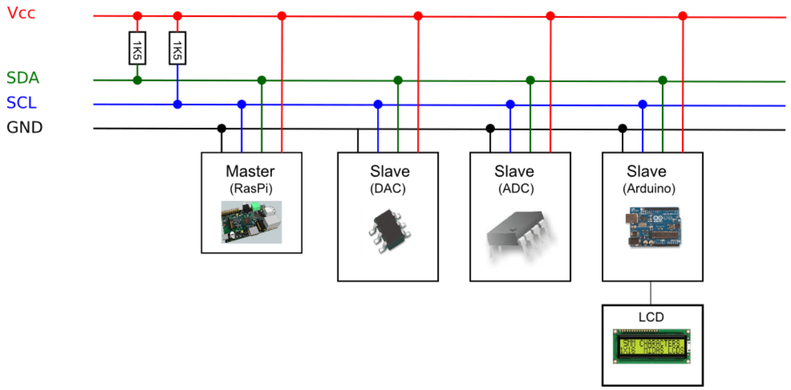


I2C - Revisão

✓ As mensagens são divididas me dois tipos de quadro (frame):

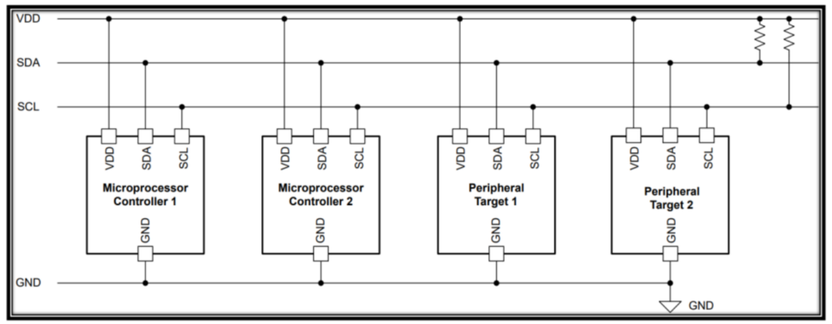
1. **Quadro de endereço**: o mestre indica o escravo para o qual a mensagem está sendo enviada. (7 bits, resulta em, no máximo, 127 dispositivos endereçáveis.

2. **Quadro de dados**: mensagens de 8 bits passadas de mestre para escravo ou vice-versa.



✓ Ambas as linhas são bidirecionais e possuem resistores de pull-up, mantendo o nível lógico alto quando o barramento está ocioso.

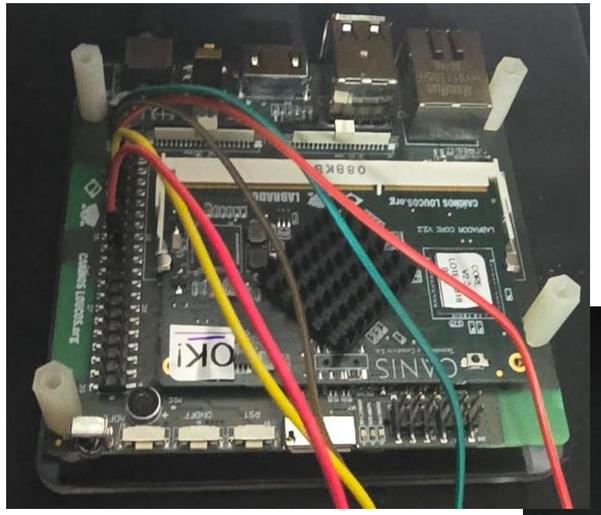
✓ O barramento é *open-drain/open-collector*, o que permite que múltiplos dispositivos compartilhem as mesmas linhas sem conflito de nível lógico.

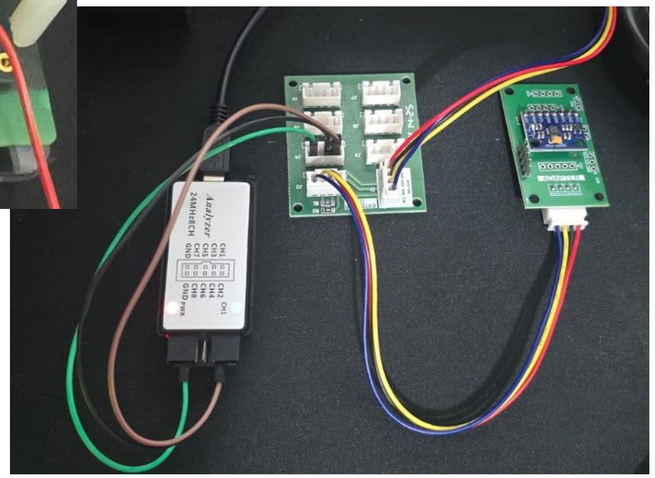


I2C

- **Labrador:**

✓ Na placa labrador, pinos I2C estão disponíveis através dos conectores GPIO (headers de 40 pinos).





Localização dos pinos:

FUNC	SODIMM	HEADER		SODIMM	FUNC
3.3V	N/C	1	2	N/C	5V
GPIOE3/TWI_SDATA	41	3	4	N/C	5V
GPIOE2/TWI2_SCLK	43	5	6	N/C	GND
GPIOB18/OAP/TS_CLK/LCD0_D19	134	7	8	17	**GPIOC27**/SPI1_SS/I2C0_SCLK/SPDIF/UART0_TX
GND	N/C	9	10	19	**GPIOC26**/SPI1_MISO/I2C0_SDATA/UART0_RX
GPIOC0/DSI_DP3/SD1_CLK/LCD0_D16	161	11	12	34	**GPIOB8**/PWM3/SD0_CLK/KS_OUT1
GPIOC1/DSI_DN3/SD1_D3/LCD0_D9	163	13	14	N/C	GND
GPIOC4/DSI_CP/SD1_D1/LCD0_D1	167	15	16	103	**GPIOA25**/SIRQ1
3.3V	N/C	17	18	149	**GPIOC6**/DSI_DP0/UART2_RX/SPI0_MISO
GPIOC25/TWI3_SDATA/PCM0_SYNC/SPI0_MOSI	53	19	20	N/C	GND
GPIOC24/I2S_MCLK1/PCM0_IN/SPI0_MISO	55	21	22	169	**GPIOC5**/LCD0_D0/DSI_CN/SD1_D0
GPIOC22/TWI3_SCLK/PCM0_CLK/SPI0_SCLK	47	23	24	51	**GPIOC23**/I2S_LRCLK1/PCM0_OUT/SPI0_SS
GND	N/C	25	26	136	**GPIOB19**/OAN/TS_START/LCD0_D15
GPIOB16/OBP/TS_IN1/LCD0_D21	140	27	28	146	**GPIOB14**/OCP/TS_IN0/LCD0_D23
GPIOB15/OCN/TS_IN2/LCD0_D22	148	29	30	N/C	GND
GPIOB10/OEP/TS_IN7/LCD0_DCLK0	158	31	32	154	**GPIOB13**/ODN/TS_IN4/LCD0_VSYNC0
GPIOB0/PCM0_OUT/I2S_BCLK1	9	33	34	N/C	GND
GPIOB1/PCM0_CLK/I2S_LRCLK1	13	35	36	7	**GPIOA28**/PCM0_IN/I2S_BCLK0
GPIOB2/PCM0_SYNC/I2S_MCLK1	15	37	38	5	**GPIOA31**/I2S_D1
GND	N/C	39	40	3	**GPIOA27**/I2S_D0

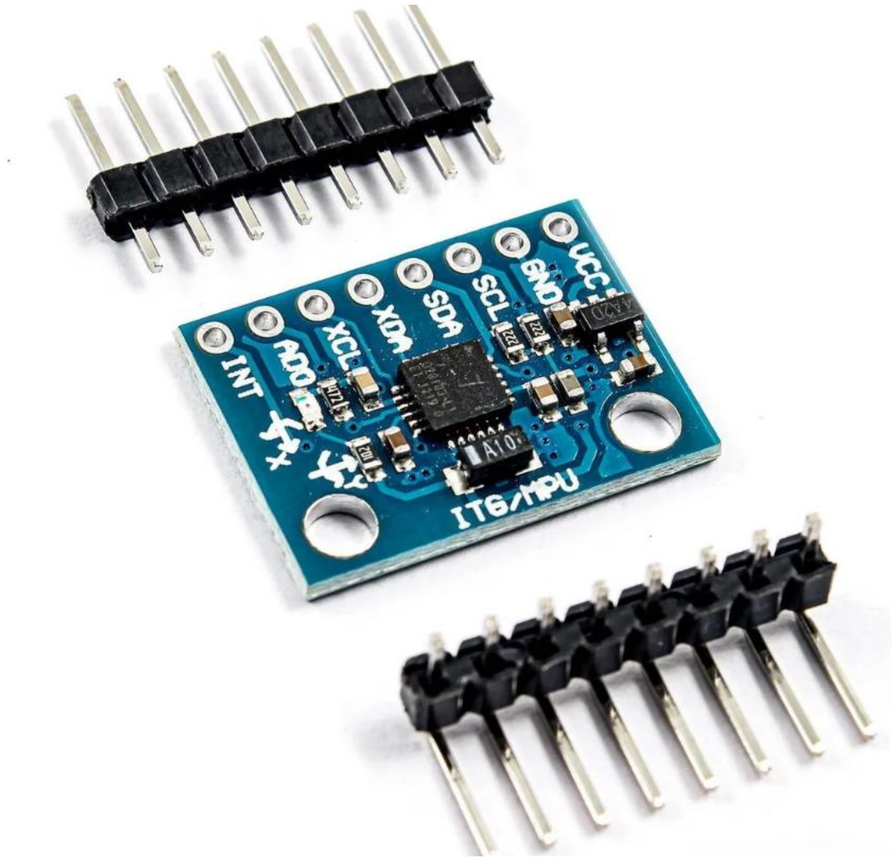
I2C

- **Labrador – comunicação com módulo I2C:**

✓ Na placa labrador, pinos I2C estão disponíveis através dos conectores GPIO (headers de 40 pinos).

✓ **Objetivo**: comunicação com o módulo MPU6050 – dispositivo que combina um acelerômetro e um giroscópio de 3 eixos em um único chip.

✓ Utiliza o protocolo I2C para troca de dados, com pinos dedicados para Clock (SCL) e Data (SDA).

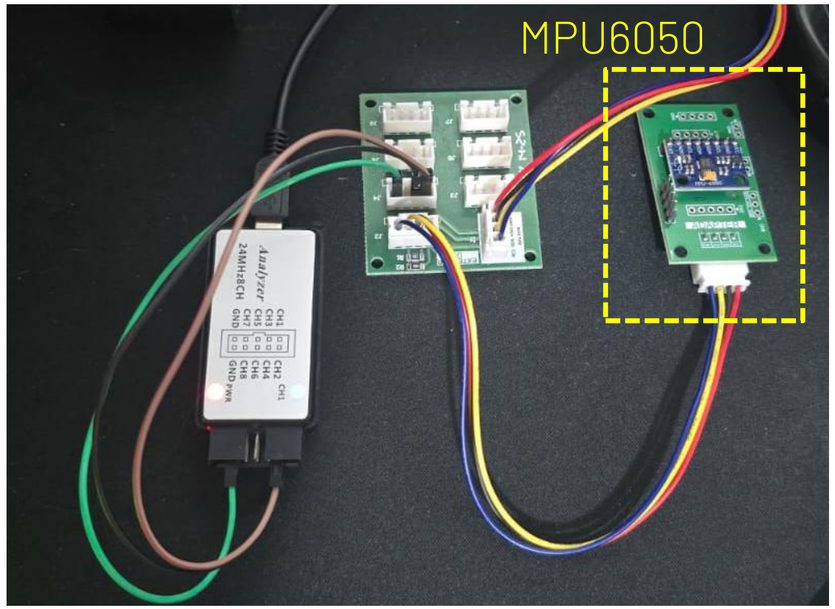


I2C

- **Labrador – comunicação com módulo I2C:**

✓ Etapas:

1. **Abrir o barramento**: i2c_init() - I²C: /dev/i2c-2;
2. **Acordar sensor**: i2c_write() -> 0x6B 0x00 - Sai do modo de "sleep";
3. **Leitura**: i2c_read_bytes() 0x3B — Lê 6 bytes de aceleração;
4. **Conversão**: (buf[0]<<8)|buf[1] — Junta bytes alto e baixo



I2C

Código em C – parte 1/3:

```c
#include <stdio.h>              // Funções de entrada/saída
#include <stdlib.h>             // Funções utilitárias (exit, malloc)
#include <fcntl.h>              // Controle de arquivos (open, O_RDWR, O_NOCTTY, etc.)
#include <unistd.h>             // Funções POSIX (read, write, close)
#include <sys/ioctl.h>          // Habilita o uso da syscall ioctl() (Input/Output Control), que envia comandos específicos a drivers de dispositivos.
#include <linux/i2c-dev.h>      // Fornece definições, constantes e macros específicas para o driver de dispositivos I²C do Linux.
#include <linux/i2c.h>          // Complementa <linux/i2c-dev.h>, trazendo estruturas e flags internas do protocolo I²C.
#include <stdint.h>             // Define tipos de inteiros com tamanhos explícitos e portáveis, evitando ambiguidades entre arquiteturas (32 bits, 64 bits etc.).

// Definições iniciais - variáveis globais e macros
typedef unsigned char u8;
int i2c_fd = -1;
const char *i2c_fname = "/dev/i2c-2"; // barramento I2C da Labrador

// Abre o barramento I²C em modo leitura/escrita (O_RDWR).
int i2c_init(void) {
    if ((i2c_fd = open(i2c_fname, O_RDWR)) < 0) {
        perror("Erro ao abrir I2C");
        return -1;
    }
    return i2c_fd;
}

// Envia dois bytes (endereço do registrador e valor) para um dispositivo escravo no barramento I²C.
int i2c_write(u8 slave_addr, u8 reg, u8 data) {
    u8 outbuf[2] = {reg, data};
    struct i2c_msg msg;
    struct i2c_rdwr_ioctl_data msgset;

    // Configuração da mensagem I2C
    msg.addr  = slave_addr;
    msg.flags = 0;
    msg.len   = 2;
    msg.buf   = outbuf;

    // Agrupamento da transação
    msgset.msgs  = &msg;
    msgset.nmsgs = 1;

    // Execução da operação via ioctl()
    if (ioctl(i2c_fd, I2C_RDWR, &msgset) < 0) {
        perror("Erro no write");
        return -1;
    }
    return 0;
}
```

I2C

Código em C – parte 2/3:

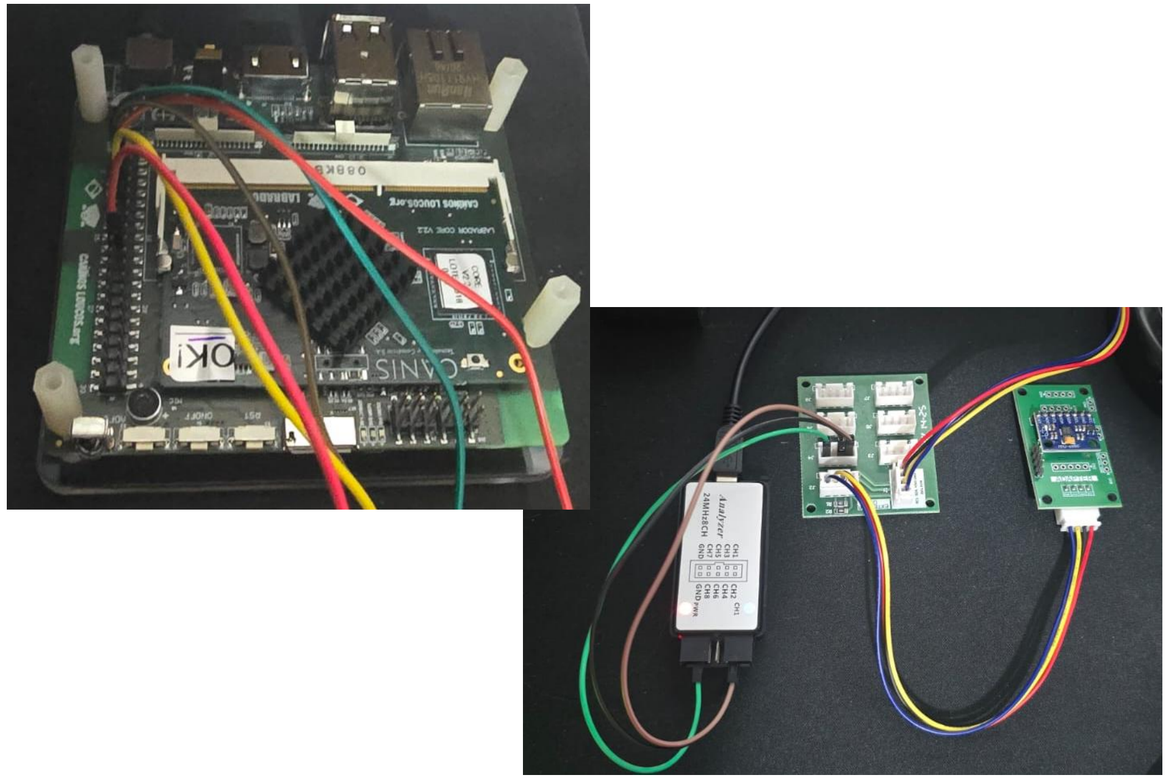
```c
// Leitura genérica de N bytes consecutivos a partir de um registrador
/*
[Start] -> [Endereço Escravo + Write] -> [Registrador] -> [Restart]
        -> [Endereço Escravo + Read]  -> [Lê N bytes] -> [Stop]
*/
int i2c_read_bytes(u8 slave_addr, u8 reg, u8 *buffer, u8 length) {
    struct i2c_msg msgs[2];
    struct i2c_rdwr_ioctl_data msgset;


    msgs[0].addr  = slave_addr;
    msgs[0].flags = 0;              // Escrita
    msgs[0].len   = 1;
    msgs[0].buf   = &reg;


    msgs[1].addr  = slave_addr;
    msgs[1].flags = I2C_M_RD;       // Leitura
    msgs[1].len   = length;
    msgs[1].buf   = buffer;


    msgset.msgs  = msgs;
    msgset.nmsgs = 2;


    if (ioctl(i2c_fd, I2C_RDWR, &msgset) < 0) {
        perror("Erro na leitura I2C");
        return -1;
    }


    return 0;
}
```

I2C

Código em C – parte 3/3:

```c
// Função principal
int main(void) {


    u8 slave_addr = 0x68;       // Endereço do MPU6050
    u8 buf[6];                  // Vetor para armazenar os 6 bytes de aceleração;
    int16_t ax, ay, az;         // valores inteiros de 16 bits (signed).

    // Abre barramento I²C /dev/i2c-2.
    i2c_init();

    // Escreve 0x00 no registrador 0x6B (PWR_MGMT_1) do MPU6050 — tira o sensor do modo de suspensão (“sleep mode”).
    i2c_write(slave_addr, 0x6B, 0x00);

    // Loop de leitura contínua
    while (1){

       // Lê 6 bytes a partir do registrador 0x3B (aceleração X/Y/Z)
        if (i2c_read_bytes(slave_addr, 0x3B, buf, 6) == 0) {
            // Combina bytes altos e baixos - Cada eixo tem 2 bytes (alto + baixo), combinados com bit shift (formando um inteiro de 16 bits):
            ax = (int16_t)((buf[0] << 8) | buf[1]);
            ay = (int16_t)((buf[2] << 8) | buf[3]);
            az = (int16_t)((buf[4] << 8) | buf[5]);

            // Exibição dos bits
            printf("Aceleração bruta:\n");
            printf("AX = %d\n", ax);
            printf("AY = %d\n", ay);
            printf("AZ = %d\n", az);
    }
        usleep(1000000); // rotina de temporização
    }

    close(i2c_fd);
    return 0;
}
```

I2C

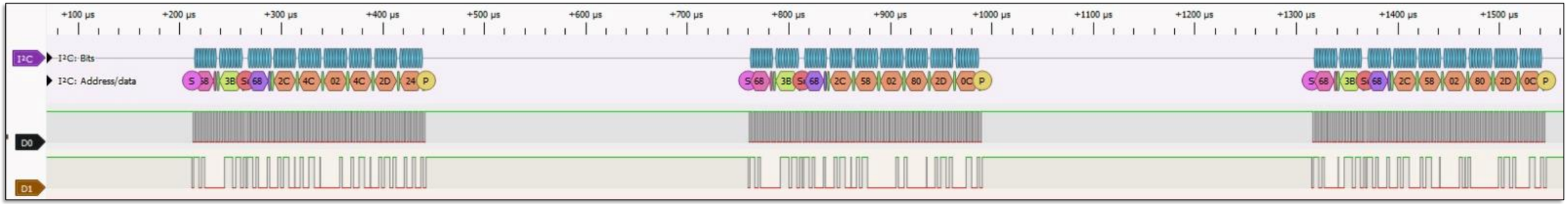


Resultados:

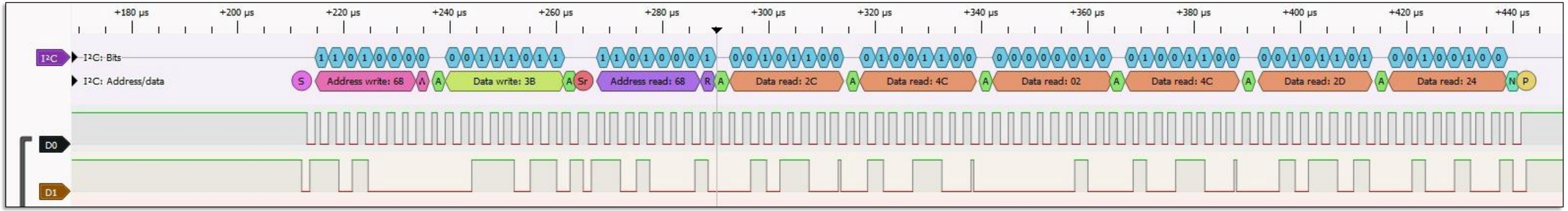
```
i2c_01
Arquivo  Editar  Abas  Ajuda
AX = -552
AY = 15932
AZ = -3516
Aceleração bruta:
AX = 1544
AY = 13948
AZ = 8068
Aceleração bruta:
AX = -76
AY = 11552
AZ = 11068
Aceleração bruta:
AX = -312
AY = 10868
AZ = 11780
Aceleração bruta:
AX = -508
AY = 8316
AZ = 13844
Aceleração bruta:
AX = 2268
AY = -484
AZ = 16892
```

I2C

Envio periódico de informações - temporização:

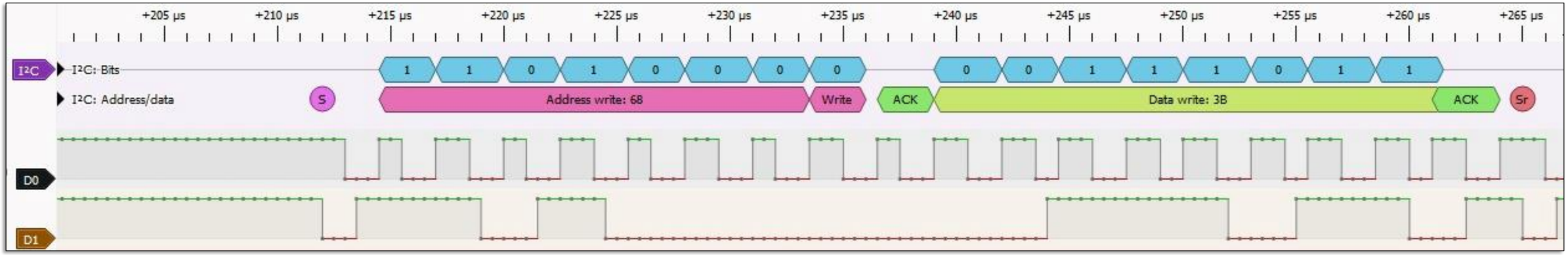


Leitura dos 6 bytes a partir do registrador 0x3B (aceleração X/Y/Z):

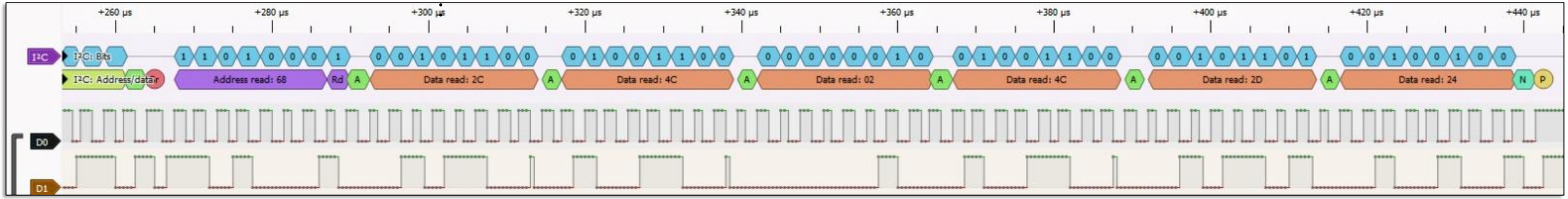


I2C

Endereço do escravo e endereço do registrador (Parte 1/2 da comunicação):



Endereço do escravo e dados de leitura (Parte 2/2 da comunicação):



Conclusões

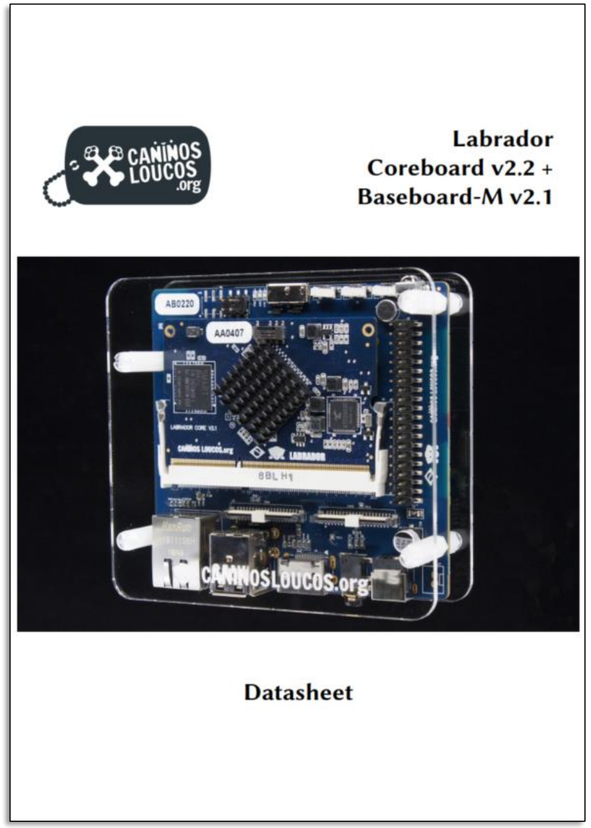
Conclusões

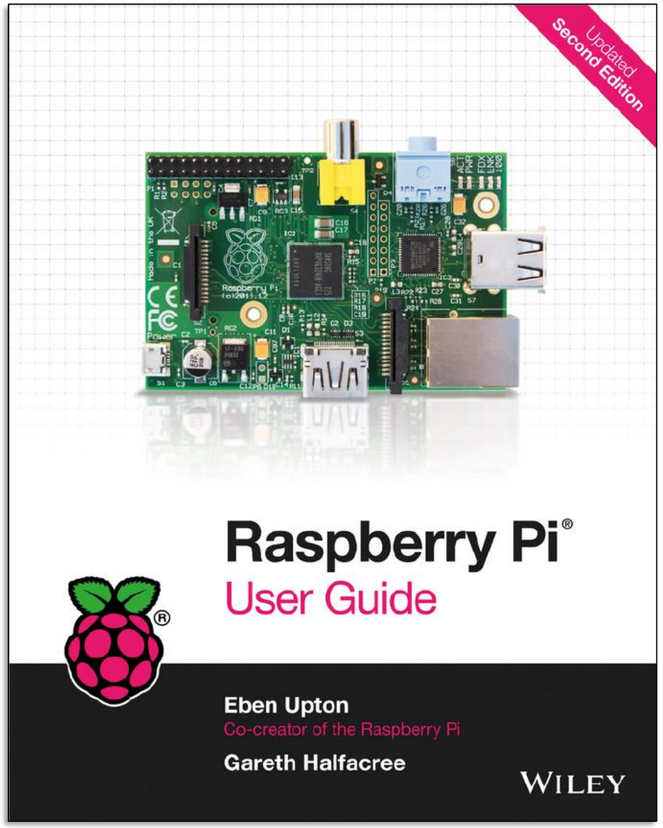
- A SBC labrador é uma ferramenta versátil e conta com periféricos que viabilizam o desenvolvimento de diversos sistemas embarcados.

- A linguagem C pode ser utilizada para realizar o controle do conversor analógico-digital (ADC) e de diferentes tipos de interfaces de comunicação serial (ex.: UART e I2C), que são periféricos disponíveis na arquitetura modular da placa labrador.

- A utilização de analisadores lógicos viabilizam a identificação do nível de conformidade dos dados digitais provenientes das interfaces de comunicação.

Bibliografia Recomendada





embarca tech | Residência Tecnológica em Sistemas Embarcados

SBC Labrador – Parte 3

RESIDÊNCIA – Aula Síncrona (14/10/2025)

Prof. Dr Ricardo Menezes Prates

Executores:
INSTITUTO FEDERAL Piauí
INSTITUTO FEDERAL Rio Grande do Norte
INSTITUTO FEDERAL Maranhão
INSTITUTO FEDERAL Ceará
cepedi
INSTITUTO HARDWARE BR
Coordenação: Softex
Iniciativa: MINISTÉRIO DA CIÊNCIA, TECNOLOGIA E INOVAÇÃO
GOVERNO FEDERAL BRASIL UNIÃO E RECONSTRUÇÃO